

DevOps na Prática na Lacrei Saúde

Guia didático de arquitetura, CI/CD, segurança, observabilidade e operação na AWS

Material de consulta diária

Projeto: Desafio Técnico DevOps — Lacrei Saúde

Autor: André Leão

Data: setembro de 2026

Como usar este e-book

Este material transforma a implementação do projeto em um guia de consulta. Ele começa pelos conceitos gerais e depois apresenta os detalhes operacionais. A ideia é ajudar a responder três perguntas no dia a dia:

1. **Como o sistema está organizado?**
2. **Como uma alteração chega com segurança aos ambientes?**
3. **Como investigar e recuperar o serviço quando algo falha?**

Os comandos usam a região `us-east-1`, o cluster `lacrei-desafio-cluster` e o repositório ECR `devops-app`. Ajuste esses valores quando estiver trabalhando em outro ambiente.

Atenção: exemplos de conta, URLs, nomes de recursos e e-mails pertencem ao ambiente deste projeto. Não copie credenciais, tokens ou valores sensíveis para o código.

1. A ideia central

A aplicação é simples: um serviço Node.js/Express com uma rota `/status`. O desafio real está em entregar essa aplicação de forma segura, repetível e operável.

A versão é construída como imagem Docker e recebe a SHA completa do commit como tag. Essa imagem é validada em staging e, depois, a mesma imagem é promovida para production. Se uma versão apresentar problema, o rollback reaplica uma imagem anterior já existente no ECR.

Código

- testes e lint
- imagem Docker
- Trivy
- smoke test
- ECR com tag imutável
- ECS staging
- health-check
- aprovação
- ECS production

A regra mais importante é:

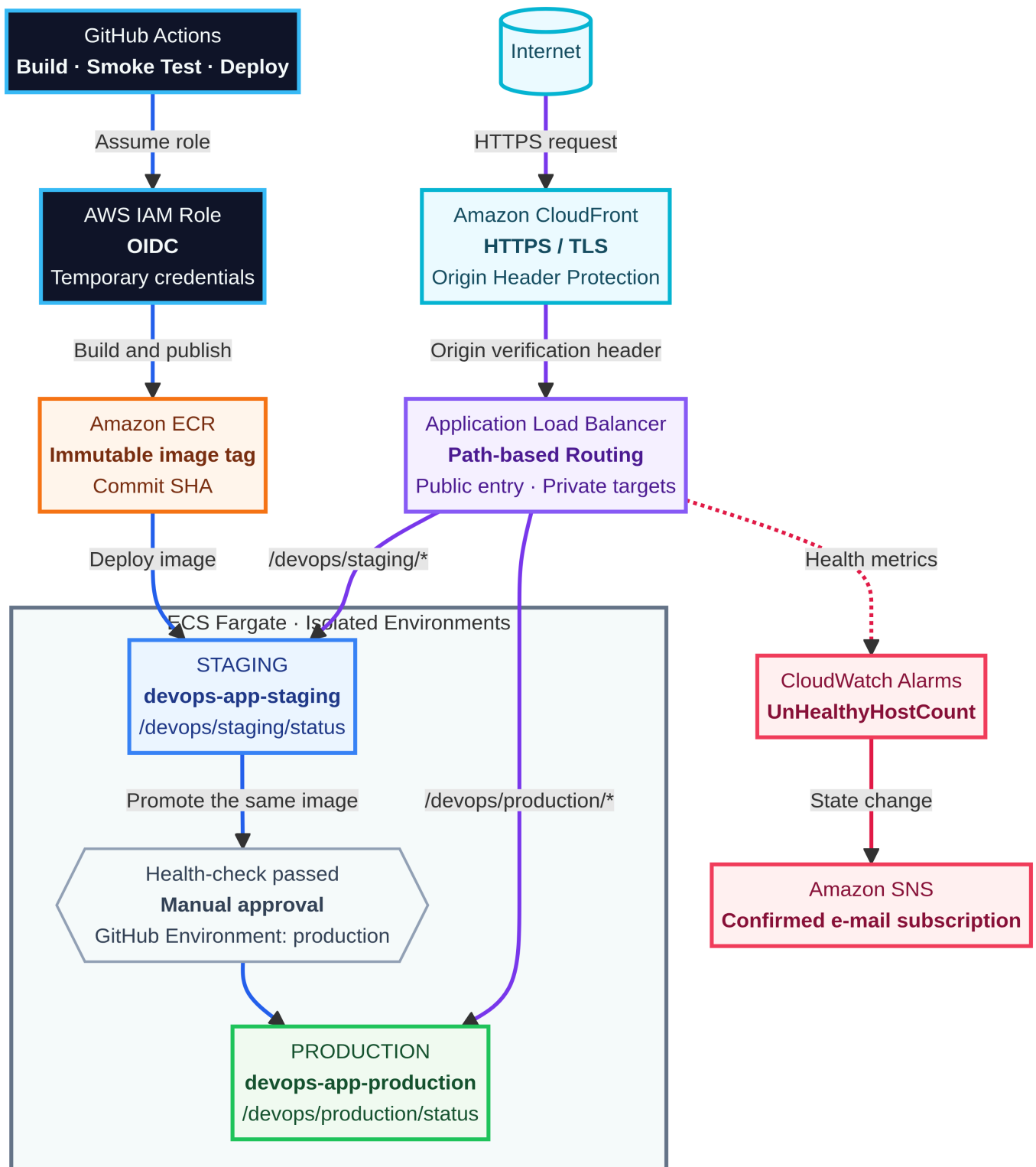
Não recompilar para production. Promover o mesmo artefato que passou por staging.

Isso melhora a rastreabilidade e reduz o risco de staging e production executarem imagens diferentes.

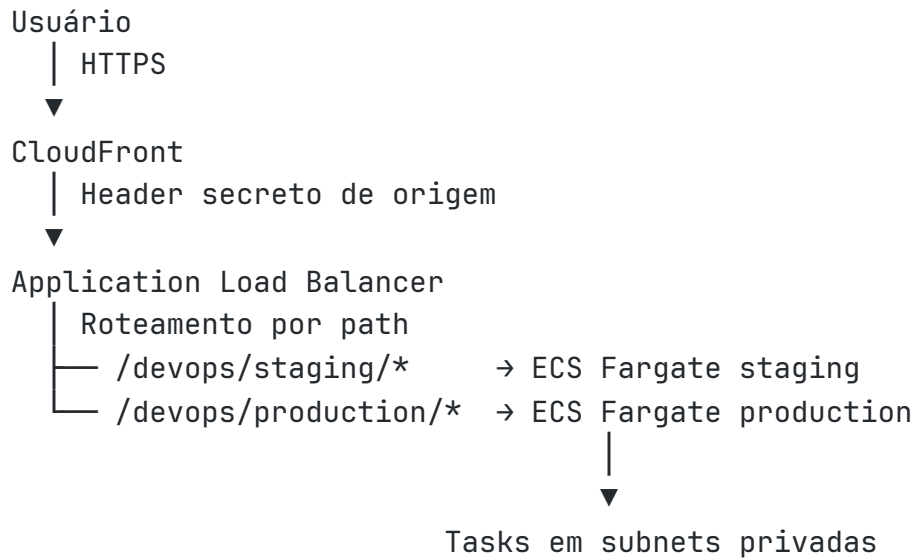
Resultados validados

Item	Resultado
Staging	HTTP 200 com status: ok
Production	HTTP 200 com status: ok
Promoção	Mesma SHA entre ambientes
Rollback	Validado em staging no run 35309972742
Alertas	SNS confirmado e e-mail recebido
CI/CD	Lint, testes, Trivy, build, deploy e health-check aprovados

2. Arquitetura completa



2.1 Caminho de uma requisição



Internet e CloudFront

O CloudFront é o endpoint público. Ele entrega HTTPS e redireciona requisições HTTP para HTTPS. Nesta entrega, os métodos permitidos são `GET` e `HEAD`, suficientes para uma aplicação de status.

Application Load Balancer

O ALB recebe o tráfego do CloudFront e encaminha a requisição ao target group correto. O path diferencia os ambientes:

```
/devops/staging/status
/devops/production/status
```

O listener também exige um header secreto enviado pelo CloudFront. Isso reduz o bypass direto do ALB, embora não seja equivalente a tornar o ALB privado.

ECS Fargate

As tasks executam em subnets privadas e não recebem IP público. O Security Group da aplicação aceita tráfego somente do Security Group do ALB.

Production mantém duas tasks desejadas. Isso permite que uma task continue atendendo enquanto a outra é reiniciada ou atualizada, desde que os demais componentes estejam saudáveis.

ECR

O ECR armazena as imagens Docker. O repositório usa tags imutáveis. Uma tag de commit já existente não pode ser sobrescrita.

Terraform

Terraform descreve a infraestrutura como código. Os módulos principais são:

- `network` : VPC, subnets, rotas e NAT;
- `ecs-cluster` : cluster, ECR e ALB;
- `ecs-service` : target groups, tasks e serviços ECS;
- `cloudfront` : distribuição e WAF;
- `alerts` : KMS, SNS e alarmes;
- `github-oidc` : provider OIDC, roles e políticas.

2.2 Limitações conscientes

O origin CloudFront → ALB usa HTTP nesta entrega porque o ALB utiliza o domínio padrão `elb.amazonaws.com` e não possui um certificado ACM associado a domínio próprio. Em uma arquitetura produtiva com domínio controlado, a evolução recomendada é habilitar HTTPS no ALB e usar `origin_protocol_policy = "https-only"`.

O projeto usa um NAT Gateway único para reduzir o custo do ambiente de demonstração. Production possui duas tasks, mas a saída das subnets privadas não possui redundância completa de NAT.

3. A aplicação e o container

A aplicação está em `app/server.js` e possui duas rotas:

Método	Rota	Função
GET	/	Identificação simples da aplicação
GET	/status	Estado, uptime e timestamp

Exemplo:

```
{  
  "status": "ok",  
  "uptime_seconds": 315.607,  
  "timestamp": "2026-09-15T02:46:44.878Z"  
}
```

Execução local

```
cd app  
npm ci  
npm run lint  
npm test  
npm start
```

Em outro terminal:

```
curl -i http://localhost:3000/status
```

Docker

```
cd app
docker build -t lacrei-status-app .
docker run --rm -p 3000:3000 lacrei-status-app
curl -i http://localhost:3000/status
```

O Dockerfile usa Node Alpine, instala dependências de runtime com `npm ci --omit=dev`, executa como usuário não-root e possui `HEALTHCHECK` baseado em `/status`.

O que os testes protegem

Os testes Jest/Supertest verificam:

- código HTTP `200` ;
- campo `status` ;
- presença do uptime;
- timestamp;
- rota raiz `/` .

O lint verifica regras estáticas. Ambos são executados antes do `docker build` ; se falharem, a imagem não é publicada.

4. CI/CD passo a passo



4.1 O que acontece no deploy

O workflow `deploy.yml` é acionado por alterações em `app/**` , `scripts/**` ou no próprio workflow na branch `main` .

Etapa 1 — Checkout

O runner baixa o código do commit que disparou o workflow.

Etapa 2 — OIDC

O GitHub Actions recebe um token OIDC temporário. A AWS valida o token contra a trust policy e permite assumir a role autorizada.

Não são usadas access keys permanentes no repositório.

Etapa 3 — Qualidade

```
npm ci
npm run lint
npm test
```

Essa ordem falha cedo e evita criar uma imagem a partir de código que já não passou pelas verificações básicas.

Etapa 4 — Build

A imagem recebe a SHA completa do commit:

```
${{ github.sha }}
```

Etapa 5 — Trivy

O Trivy analisa a imagem para vulnerabilidades **HIGH** e **CRITICAL**. O pipeline falha quando a política configurada encontra um problema bloqueante.

O workflow de infraestrutura também executa Trivy sobre a configuração Terraform.

Etapa 6 — Smoke test

O pipeline inicia o container e chama:

```
http://localhost:3000/devops/staging/status
```

Isso verifica se a imagem inicia e se a aplicação responde antes do push.

Etapa 7 — Push idempotente

O pipeline consulta o ECR antes do push:

```
aws ecr describe-images \  
  --repository-name "$ECR_REPOSITORY" \  
  --image-ids imageTag="${GITHUB_SHA}"
```

Se a tag já existir, a imagem é reutilizada. Essa lógica evita falha ao reexecutar um workflow parcial em um ECR imutável.

Etapa 8 — Deploy staging

`scripts/deploy-ecs.sh` lê a task definition, troca a imagem, registra nova revision, atualiza o serviço e aguarda `services-stable`.

Etapa 9 — Health-check staging

O workflow chama a variável `STAGING_URL` até obter sucesso. Se staging falhar, production não é executado.

Etapa 10 — Aprovação

O job de produção usa o GitHub Environment `production`, que pode exigir required reviewer.

Etapa 11 — Promoção

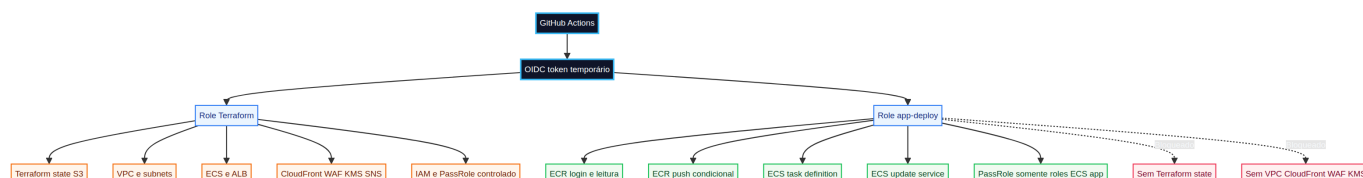
Production recebe exatamente a mesma SHA validada em staging. Não há segundo build.

4.2 Workflows e responsabilidades

Workflow	Responsabilidade	Role
<code>infrastructure.yml</code>	fmt, validate, Trivy, plan e apply controlado	Terraform
<code>deploy.yml</code>	testes, build, ECR, staging e production	app-deploy
<code>rollback.yml</code>	reaplicar SHA existente no ECR	app-deploy

O workflow de deploy não executa Terraform. Isso reduz o impacto de um erro no pipeline da aplicação.

5. IAM, OIDC e Least Privilege



Por que separar as roles?

Terraform precisa criar e alterar a plataforma. Deploy precisa apenas publicar a imagem e atualizar ECS. Se os dois usassem a mesma role, um pipeline de aplicação poderia alterar VPC, WAF, CloudFront, KMS ou o state Terraform.

Role de infraestrutura

`lacrei-desafio-github-actions` é usada por `infrastructure.yml`. Ela é mais ampla porque precisa operar:

- VPC e subnets;
- ECS, ALB e ECR;
- CloudFront e WAF;
- KMS, SNS e CloudWatch;
- IAM;
- state remoto do Terraform.

Role de aplicação

`lacrei-desafio-app-deploy` é usada por deploy e rollback. Seu escopo é ECR e ECS da aplicação. Ela não deve acessar:

- state S3;

- VPC;
- CloudFront;
- WAF;
- KMS;
- recursos de infraestrutura não necessários.

`iam:PassRole` é limitado às roles de execução ECS da aplicação.

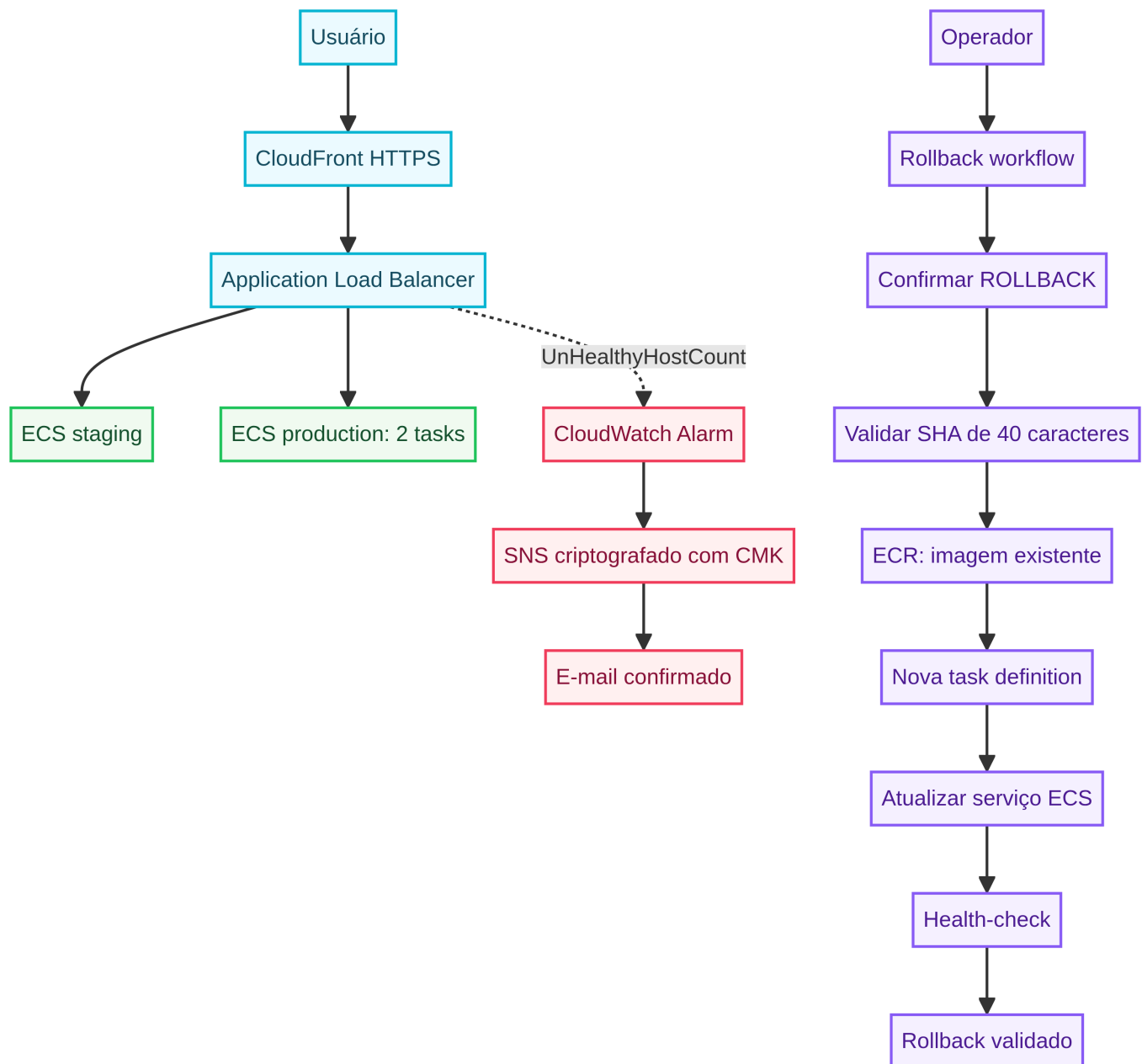
Least privilege não significa que toda role tenha poucas permissões. Significa que cada identidade possui somente os poderes necessários para a responsabilidade que executa.

Trust policy e Environments

Um job que declara `environment: production` possui um subject OIDC diferente de um job executado diretamente na branch. Por isso a trust policy deve autorizar os subjects necessários para `main`, `staging` e `production`.

Esse detalhe causou uma falha real no primeiro teste de rollback e foi corrigido antes da validação final.

6. Observabilidade e alarmes



6.1 Métrica principal

O CloudWatch monitora:

```
Namespace: AWS/ApplicationELB
Métrica: UnHealthyHostCount
```

Existe um alarme para cada ambiente:

Configuração	Valor
Período	60 segundos
Períodos de avaliação	2
Estatística	Maximum
Limite	≥ 1 target unhealthy
Dados ausentes	breaching
Entrada em alarme	SNS
Retorno a OK	SNS

As dimensões restringem a métrica ao ALB e ao target group do ambiente. Isso evita misturar staging com production.

6.2 Health check do target group

Parâmetro	Valor
Rota staging	/devops/staging/status
Rota production	/devops/production/status
Intervalo	15 segundos
Timeout	5 segundos
Healthy threshold	2 respostas
Unhealthy threshold	3 falhas
Matcher	HTTP 200

O target group decide se cada target é saudável. O alarme observa quantos targets unhealthy existem no agregado.

6.3 Estados

Estado	Significado	Ação
OK	Nenhum target unhealthy na janela.	Continuar monitorando.
ALARM	Pelo menos um target unhealthy em dois períodos.	Investigar e considerar rollback se a versão for a causa.
INSUFFICIENT_DATA	Datapoints insuficientes ou métrica indisponível.	Verificar ALB, target group, tasks e publicação da métrica.

`treat_missing_data = "breaching"` torna o monitoramento conservador: ausência de dados não é considerada automaticamente segura.

6.4 O que está sendo monitorado

Configurado:

- targets unhealthy do ALB;
- health check HTTP;
- logs da aplicação em CloudWatch Logs;
- estabilidade do serviço ECS durante deploy;
- notificações de estado via SNS.

Não configurado como alarme nesta entrega:

- CPU;
- memória;
- latência do ALB ou CloudFront.

Não se deve afirmar que esses últimos sinais são alarmados. Eles podem ser adicionados em uma evolução de capacidade, autoscaling ou SLO.

6.5 Caminho do alerta

```
Target unhealthy
  ↓
Métrica do ALB
  ↓
Alarme CloudWatch
  ↓
SNS criptografado com CMK
  ↓
E-mail confirmado
```

6.6 Runbook de investigação

Quando chegar um alerta:

1. Identifique staging ou production no nome do alarme.
2. Consulte o estado e o motivo.
3. Consulte a saúde dos targets.
4. Verifique desired, running e pending count do ECS.
5. Consulte os eventos recentes do serviço.
6. Consulte os logs da aplicação.
7. Teste o endpoint público.
8. Compare o horário com o último deploy.
9. Faça rollback somente se a evidência indicar que a versão é a causa.

Comandos úteis:

```
AWS_PROFILE=lacrei-desafio aws cloudwatch describe-alarms \
  --alarm-name-prefix lacrei-desafio \
  --region us-east-1 \
  --query 'MetricAlarms[*].[AlarmName,StateValue,StateReason]' \
  --output table
```

```
AWS_PROFILE=lacrei-desafio aws elbv2 describe-target-health \
  --target-group-arn <TARGET_GROUP_ARN> \
  --region us-east-1 \
  --query 'TargetHealthDescriptions[*].[Target.Id,TargetHealth.State,TargetHealth.DesiredCount]' \
  --output table
```

```
AWS_PROFILE=lacrei-desafio aws ecs describe-services \
  --cluster lacrei-desafio-cluster \
  --services lacrei-desafio-devops-app-staging lacrei-desafio-devops-app-production \
  --region us-east-1 \
  --query 'services[*].[serviceName,desiredCount,runningCount,pendingCount]' \
  --output table
```

7. Rollback por SHA

O rollback oficial reaplica uma imagem existente. Ele não recompila e não executa Terraform.

Passo a passo

1. Liste as imagens do ECR.
2. Escolha uma SHA estável de 40 caracteres.
3. Abra **Actions → Rollback DevOps App → Run workflow**.
4. Selecione **staging** ou **production**.
5. Informe a SHA.
6. Digite exatamente **ROLLBACK**.
7. O workflow valida formato e existência da imagem.
8. O script registra uma task definition apontando para a imagem.
9. ECS é atualizado e aguarda estabilidade.
10. O endpoint é validado.

Listar imagens:

```
AWS_PROFILE=lacrei-desafio aws ecr describe-images \
  --repository-name devops-app \
  --region us-east-1 \
  --query 'sort_by(imageDetails,& imagePushedAt)[-10:].[imageTags[0],imagePushedAt]' \
  --output table
```

Executar pelo CLI:

```
gh workflow run rollback.yml \
  --repo LeaoAndre3107/Desafio_Tecnico_Lacrei_Saude \
  --ref main \
  -f environment=staging \
  -f image_tag=5438c8569dd477faadde95f08a4662e40ed72fc7 \
  -f confirm=ROLLBACK
```

Validar:

```
curl -i https://d1gjy0g4ibj9zk.cloudfront.net/devops/staging/status
```

Evidência validada:

```
Run: 35309972742
Ambiente: staging
Resultado: sucesso
```

Rollback emergencial direto no ECS

Em uma indisponibilidade que exija ação imediata, é possível apontar o serviço para uma revision anterior de task definition. Esse procedimento é separado do workflow oficial e deve ser seguido por uma correção no código ou pipeline.

```
AWS_PROFILE=lacrei-desafio aws ecs list-task-definitions \
  --family-prefix lacrei-desafio-devops-app-staging \
  --status ACTIVE \
  --sort DESC \
  --region us-east-1

AWS_PROFILE=lacrei-desafio aws ecs update-service \
  --cluster lacrei-desafio-cluster \
  --service lacrei-desafio-devops-app-staging \
  --task-definition <REVISION_ESTAVEL> \
  --region us-east-1
```

8. Problemas reais e como diagnosticá-los

Push em tag imutável

Sintoma: `docker push` falha porque a tag já existe.

Causa: uma execução parcial já publicou a imagem.

Solução: consultar o ECR antes do push e reutilizar a tag.

Lint não encontra configuração

Sintoma: ESLint informa que não encontrou arquivo de configuração.

Causa: a configuração estava fora do diretório `app`.

Solução: manter `app/.eslintrc.json` junto do código e do `package.json`.

Script ECS sem permissão

Sintoma: exit code `126` ou `Permission denied`.

Causa: o script não estava executável no checkout.

Solução: versionar os scripts com modo `755` e conferir com `ls -l`.

```
ls -l scripts/*.sh
```

OIDC não assume role

Sintoma: `Not authorized to perform sts:AssumeRoleWithWebIdentity` .

Causa: subject OIDC do GitHub Environment não estava na trust policy.

Solução: conferir owner ID, repository ID, branch e Environments autorizados.

Health-check falha apesar de URL funcionar localmente

Sintoma: `curl` funciona manualmente, mas o workflow falha.

Causa: variável GitHub armazenada como `STAGING_URL=https:// ...` em vez de somente `https:// ...` .

Solução: o campo Name contém `STAGING_URL` ; o campo Value contém apenas a URL.

Terraform lock

Sintoma: `Error acquiring the state lock` .

Causa: processo interrompido ou outra operação usando o state.

Solução: verificar processos e workflows antes de `force-unlock` . Não usar `-lock=false` como atalho.

Destroy com identidade errada

Sintoma: `kms:DescribeKey` ou `wafv2:GetWebACL` negado.

Causa: `terraform destroy` foi executado com o usuário restrito de deploy.

Solução: usar uma identidade de infraestrutura, sem ampliar a role de deploy.

Verificar identidade:

```
AWS_PROFILE=lacrei-desafio aws sts get-caller-identity
```

A role de deploy deve continuar sem permissões de infraestrutura. O erro, nesse caso, é evidência de que o Least Privilege está funcionando.

9. Operação diária

Verificar endpoints

```
curl -i https://d1gjy0g4ibj9zk.cloudfront.net/devops/staging/status
curl -i https://d1gjy0g4ibj9zk.cloudfront.net/devops/production/status
```

Verificar serviços ECS

```
AWS_PROFILE=lacrei-desafio aws ecs describe-services \
  --cluster lacrei-desafio-cluster \
  --services lacrei-desafio-devops-app-staging lacrei-desafio-devops-app-l
  --region us-east-1 \
  --query 'services[*].[serviceName,desiredCount,runningCount,pendingCount]' \
  --output table
```

Verificar alarmes

```
AWS_PROFILE=lacrei-desafio aws cloudwatch describe-alarms \
  --alarm-name-prefix lacrei-desafio \
  --region us-east-1 \
  --query 'MetricAlarms[*].[AlarmName,StateValue,StateReason]' \
  --output table
```

Verificar último deploy

```
gh run list \  
  --repo LeaoAndre3107/Desafio_Tecnico_Lacrei_Saude \  
  --limit 10
```

Ver detalhes de um run:

```
gh run view <RUN_ID> \  
  --repo LeaoAndre3107/Desafio_Tecnico_Lacrei_Saude \  
  --verbose
```

Ver somente a falha:

```
gh run view <RUN_ID> \  
  --repo LeaoAndre3107/Desafio_Tecnico_Lacrei_Saude \  
  --log-failed
```

Terraform com segurança

```
cd Terraform  
export TF_VAR_alert_email="seu-email@example.com"  
AWS_PROFILE=PROFILE_INFRA terraform fmt -check -recursive  
AWS_PROFILE=PROFILE_INFRA terraform init  
AWS_PROFILE=PROFILE_INFRA terraform validate  
AWS_PROFILE=PROFILE_INFRA terraform plan
```

Use uma identidade de infraestrutura para Terraform. A identidade de deploy da aplicação não deve ler ou destruir KMS, WAF, VPC ou state.

10. Checklist de segurança

Controle	Situação
OIDC em vez de access keys permanentes	Aplicado
Role Terraform separada da role app-deploy	Aplicado
<code>iam:PassRole</code> limitado às roles ECS	Aplicado
ECR com tags imutáveis	Aplicado
Tasks sem IP público	Aplicado
Security Group das tasks limitado ao ALB	Aplicado
CloudFront com HTTPS	Aplicado
CloudFront somente <code>GET</code> e <code>HEAD</code>	Aplicado
WAF associado ao CloudFront	Aplicado
Trivy em Terraform e imagem	Aplicado
Lint e testes antes do build	Aplicado
Duas tasks em production	Aplicado
SNS com CMK dedicada	Aplicado
Alertas de target unhealthy	Aplicado
<code>treat_missing_data = "breaching"</code>	Aplicado
Rollback por SHA validado	Aplicado

11. Glossário rápido

ALB: Application Load Balancer. Distribui requisições entre targets.

AMI: não utilizada diretamente neste projeto; é uma imagem de máquina EC2.

ECR: Elastic Container Registry. Armazena imagens Docker.

ECS: Elastic Container Service. Orquestra containers.

Fargate: modo serverless de execução de tasks ECS.

Health check: verificação automática para saber se um target responde corretamente.

IAM: Identity and Access Management. Controla identidades e permissões AWS.

OIDC: protocolo usado para permitir que GitHub Actions assuma uma role temporária.

S3 backend: armazenamento remoto do state Terraform.

SNS: serviço de publicação de notificações.

Target: endereço registrado no target group do ALB, neste caso uma task Fargate.

Task definition: especificação de como o ECS executa um container.

Trivy: scanner de vulnerabilidades e misconfigurações.

WAF: Web Application Firewall.

12. Referências técnicas

Encerramento

Este projeto demonstra que DevOps não é apenas executar um deploy. É construir um caminho confiável entre uma alteração de código e um serviço funcionando, com controle de acesso, evidência de validação, observabilidade e uma forma segura de voltar atrás.